

ETL and Data Extraction in Power BI

A training handout for data analytics learners — covering the ETL process, Power Query, data extraction methods, and best practices.

1. What is ETL?

ETL stands for **Extract, Transform, Load** — the process of moving raw data from one or more source systems into a destination system (like a data warehouse, dataset, or reporting tool) in a clean, structured, and usable form.

- **Extract** — Pull raw data from source systems (databases, files, APIs, cloud services, web pages).
- **Transform** — Clean, reshape, filter, merge, and enrich the data so it is analysis-ready.
- **Load** — Deliver the transformed data into the target model or storage layer for reporting.

In Power BI, the ETL process is largely handled by **Power Query**, Microsoft's data connectivity and transformation engine built into Power BI Desktop, Excel, and Dataflows.

2. How ETL Maps to Power BI

ETL Stage	Power BI Component	What Happens
Extract	Get Data / Power Query connectors	Connect to a source and pull data into the query editor
Transform	Power Query Editor (M language)	Clean, reshape, merge, and shape the data
Load	Load to Data Model	Data is loaded into the semantic model for use in visuals/DAX

3. Data Extraction in Power BI ('Get Data')

Extraction is the first step: connecting Power BI to a data source and pulling data in. This is done through the **Get Data** dialog in Power BI Desktop.

3.1 Common Data Sources

Category	Examples
Files	Excel, CSV/Text, PDF, XML, JSON, Folder (multiple files)
Databases	SQL Server, Oracle, MySQL, PostgreSQL, Snowflake, Azure SQL
Online Services	SharePoint, Dynamics 365, Salesforce, Google Analytics, Azure services

Web / API	Web pages (HTML tables), REST APIs (via Web connector or JSON)
Other	OData feed, Python/R scripts, Blank Query (manual M code)

3.2 Steps to Extract Data

- Open Power BI Desktop → **Home** tab → click **Get Data**.
- Choose the source category (e.g. Excel workbook, SQL Server database, Web).
- Provide connection details (file path, server/database name, URL, or credentials).
- Select the specific table(s), sheet(s), or query result to bring in, using the **Navigator** window.
- Click **Transform Data** to open Power Query Editor (recommended) instead of loading directly, so the data can be cleaned before it reaches the model.

3.3 Extraction Modes

Mode	Description
Import	Data is copied into Power BI's in-memory model. Fast, most common for training/reporting.
DirectQuery	Power BI queries the source live, without importing. Used for very large or real-time data.
Live Connection	Connects directly to an existing Analysis Services/Power BI dataset model.

4. Transformation — Power Query Editor

Once data is extracted, it lands in the **Power Query Editor**, where every transformation step is recorded in the **Applied Steps** pane (right side) — this is Power BI's visual, no-code way of writing ETL logic, translated behind the scenes into the M language.

4.1 Common Transformation Operations

- **Remove/Keep Columns** — drop unused fields or keep only relevant ones.
- **Remove Duplicates / Remove Errors** — clean up rows.
- **Change Data Type** — set columns to Text, Whole Number, Decimal, Date, etc.
- **Split Column** — by delimiter, number of characters, or positions.
- **Merge Queries** — join two tables (like SQL JOIN) based on a common key.
- **Append Queries** — stack rows from multiple tables with the same structure.
- **Group By** — aggregate data (sum, count, average) by category.
- **Pivot / Unpivot Columns** — reshape wide data to long format or vice versa.
- **Filter Rows** — keep only rows matching a condition.
- **Replace Values / Fill Down** — handle blanks and inconsistent entries.
- **Add Custom Column** — create calculated columns using M formulas.

4.2 The M Language

Every action in Power Query Editor generates M code automatically. Advanced users can view/edit this through the **Advanced Editor** for full control over the transformation logic.

Example M snippet:

```
let
    Source = Excel.Workbook(File.Contents("Sales.xlsx"), null, true),
    Sales_Sheet = Source{[Item="Sales",Kind="Sheet"]}[Data],
    ChangedType = Table.TransformColumnTypes(Sales_Sheet,{"Date", type date})
in
    ChangedType
```

5. Loading Data into the Model

After transformation, clicking **Close & Apply** loads the cleaned tables into Power BI's data model, where relationships between tables are defined and DAX measures/columns are added for reporting.

- Define relationships between tables (usually Star Schema: Fact tables + Dimension tables).
- Set data types and formatting for each field.
- Organize fields into folders/hierarchies for report usability.
- Set up scheduled refresh (in Power BI Service) to re-run the ETL automatically.

6. Best Practices

- Always click **Transform Data** instead of Load — never skip the cleaning step.
- Remove unused columns and rows as early as possible to improve performance.
- Use **descriptive step names** in Applied Steps for readability (rename via right-click → Rename).
- Prefer **folding-friendly** steps (filters, column selection) early, so Power Query can push work back to the source database (Query Folding) for speed.
- Build reusable **Dataflows** when the same transformation logic is needed across multiple reports.
- Document each query's purpose with the **Query Description** field for team handoffs.

7. Quick Recap

Extract	Transform	Load
Get Data / connectors Import vs DirectQuery Navigator selection	Power Query Editor Applied Steps M language	Close & Apply Data model & relationships Scheduled refresh

Prepared as a training handout for data analytics learners.